control pattern

control pattern

definiscono l'organizzazione dei task e le loro sincronizzazioni → come le operazioni sono ordinate e eseguite nel tempo

serial control patterns

introduction

- structured serial programming forms the basis upon which parallel patterns are built → understanding these standard patterns is crucial because identifying them in serial code is often the first step in "parallelizing" an algorithm
- all structured serial programming is based on four fundamental control flow patterns
  - **sequence**
  - **selection**
  - **iteration**
  - **recursion**
- the **Nesting Pattern** allows these four patterns to be composed hierarchically → any task block within these patterns can itself be another instance of a sequence, selection

sequence pattern

this is the most fundamental structure in imperative programming

- **definition** → an ordered list of tasks that are executed in a strict, specific order
- **assumption** → the ordering defined in the program text must be followed
  - obvious in serial programming
  - in parallel programming, we constantly question the assumption *"does task B really need to wait for task A?"*
  - in parallelo, A e B possono correre insieme se non ci sono dipendenze
- **dependency analysis** → to parallelize a sequence, we analyze the dependencies (IN/OUT sets) between the steps
  - if functions depend on the output of immediately preceding functions, the sequence must remain serial
    - ex. T=f(A) S=g(T) B=h(S)
    - S depends on T from step 1 and B depends of S from step 2
  - if disjoint functions exists, the sequence can be parallelized
    - ex. T=f(A) S=g(A) B=h(S,T)

selection pattern

- it handles conditional execution → ex. if-else, switch
  - una condizione determina quale task eseguire
- **structure** → a condition c is evaluated first
  - based on the result (True/False), control flows to either task a or task b
  - if (c) {a;} else {b;}
- **key assumption** → tasks a and b are mutually exclusive → only **one** of them is executed, never both
- **parallel implication** → in parallel systems, we might sometimes execute *both* branches (a and b) concurrently while calculating the condition c in parallel
  - once c is resolved, we keep the correct result and discard the other
  - this trades extra computation (energy/resources) for lower latency
  - in parallelo si può usare la speculative selection, eseguendo entrambi i rami contemporaneamente e scartando poi il risultato inutile

iteration pattern

- it represents loops (for, while)
  - ripetizione di un task (loop)
- **structure** → a condition c is evaluated
  - if true, the body task a is executed, then c is re-evaluated → this repeats until c is false
  - for (i = 0; i < n; i++) {a;} while (c) {a;}
- **parallel challenge** → iteration is the most common source of parallelism, but also the most complex due to **loop-carried dependencies**
  - if iteration i depends on the result of iteration i-1, the loop is inherently serial
  - if iterations are independent (DOALL), the pattern can be transformed into a **Parallel Map**
- **optimization** → techiniques like **Loop Unrolling** (executing multiple iterations in a single step) are derived from analyzing this pattern

recursion pattern

- the **Recursion Pattern** involves functions calling themselves, creating dynamic nesting
  - funzioni che chiamano se stesse
- **definition** → a dynamic form of nesting where tasks spawn new instances of themselves
- **tail recursion** → a specific form of recursion where the recursive call is the final action
  - this is significant because it can be easily converted into an **iteration** pattern (loops), which is often more efficient
- **divide and conquer** → recursion is the foundation of the "Divide and Conquer" parallel strategy
  - a problem is recursively split into independent sub-problems (e.g., Quicksort, Merge, Sort)
  - since the sub-problems operate on different data, they can be solved in parallel
  - in pratica, la funzione va a richiamare se stessa su una porzione di dati più piccola effettuando il passaggio di "divisione" in modo automatico

parallel control patterns

introduction

- parallel control patterns are essentially parallel extensions of the serial control patterns
- they relax the strict ordering assumptions of serial patterns to allow concurrent execution

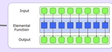
fork-join pattern

like openMP does

- the **fork-join** pattern is a dynamic control flow pattern that allows the execution to branch off and merge later
  - il flusso di controllo si divide in più flussi paralleli e successivamente si riunisce
- **mechanism**
  - **fork (spawn)** → the control flow splits into multiple parallel flows
    - in language like Cilk Plus, this is achieved via a spawn keyword → a function that spawns another function continues to execute its own code in parallel with the spawned function
  - **join (sync)** → the parallel flows merge back into a single flow → the caller waits (sync) for the spawned function to complete
- **crucial distinction: join vs barrier**
  - **join (sync)** → only the "master" or calling thread continues execution after the synchronization point
    - sincronizzazione locale
    - the spawned tasks effectively "die" or merge back
  - **barrier** → all threads reach the synchronization point, wait for each other, and then all threads continue execution
    - sincronizzazione totale

map pattern

- the **map** pattern is the most common and often the simples parallel pattern → frequently referred to as "embarrassingly parallel"
- **definition** → performs a function on every element of a collection
  - replica una funzione su ogni elemento di una funzione di input
- **structure** → it replicates a serial iteration pattern (like a for loop) where
  - the number of iterations is known
  - each iteration is completely **independent** of the others
    - non ci deve essere comunicazione tra le iterazioni
- **elemental function** → the logic applied to each data item is called the "elemental function"
  - because of independence, this function can be applies to all elements simultaneously without communication
  - complexity is O(1) but we have to consider the number of processors



stencil pattern

- the **stencil** pattern is a specific generalization of the Map pattern
- **definition** → like Map, it applies an elemental function to a grid of data
  - applichiamo una funzione a ogni elemento ma l'output dipende anche dai vicini dell'elemento corrente
  - however, unlike Map, the computation for a specific element depends on its neighbors → ex. A[i] depends on A[i-1] and A[i+1]
- **use cases** → standard pattern for
  - image processing (e.g., blurring, edge detection)
  - physical simulations (iterative solvers, fluid dynamics)
  - un altro esempio è la convoluzione in CUDA dove proviamo ad ottimizzare gli accessi alla memoria
- **boundary conditions** → a critical implementation detail in Stencil patterns is handling the boundaries (edges) of the data gris, ad these elements do not have a full set of neighbors
  - this must be handled carefully (e.g., padding, wrapping)
  - complexity is O(1) but we have to consider the number of processors

reduction pattern

- the **reduction** pattern is used to summarize a collection of data into a single value → combina tutti gli elementi di una collezione in un unico risultato riassuntivo
- **mechanism** → it combines every element in a collection using a **Combiner Function**
- **associative requirement** → to be parallelizable, the combiner function *must be* **associative**
  - i.e., (A+B)+C=A+(B+C)
  - if the operator is associative, the system can reorder the operations
    - instead of a linear sum (Serial Reduction)
    - it can perform a **Tree Reduction** (Parallel Reduction), summing pairs in parallel
      - work complexity O(N)
      - time complexity O(log(n))

- **examples**
  - addition
  - multiplication
  - maximum
  - minimum
  - boolean and/or/xor

scan pattern

- the **Scan** pattern (also known as **Prefix Sum**) computes all partial reductions of a collection
- **definition** → for every output position in a collection, it computes the reduction of all input elements up to that point
  - ex. input [1,2,3] → output [1,3,6]
  - produce una sequenza di output dove ogni elemento è la riduzione di tutti i precedenti
- **parallel paradox** → at first glance, Scan looks inherently serial (each element depends on the previous sum)
  - however, if the function is associative, it can be parallelized using a tree-based structure (Up-Sweep/Down-Sweep algorithms)
  - sembra seriale ma si può parallelizzare efficientemente con l'algoritmo up-sweep/down-sweep

- **work complexity** → a parallel scan is work-inefficient compared to serial scan
  - it performs more total operations (O(N) for serial vs roughly 2N or more for parallel)
  - it achieves lower latency (O(log N)) given enough processors
  - work complexity O(N)
  - time complexity O(log(N))

recurrance pattern

- the **recurrance** pattern is a more complex version of Map involving loop dependencies → le iterazioni dipendono le une dalle altre
- **definition** → elements can use the outputs of adjacent elements as inputs → similar to Stencil, but often implies a directional dependency, like a[i] = f(a[i-1])
- **computability** → for a recurrence to be computable, there must be a strictly defined serial ordering → while difficult to parallelize directly, specific techniques (like parallel prefix scan) can sometimes transform recurrences into parallelizable forms